

Covercrypt

An efficient ABE-like encryption scheme.

Contents

1	Introduction	2
2	Subset-cover KEM-AC	3
2.1	Key-Encapsulation Mechanism with Access Control (KEM-AC) .	3
2.2	Key and encapsulation life-cycle management	4
2.3	TODO Hybridization à la carte	6
2.4	Live access-structure modifications	6
3	Boolean KEM-AC	6
3.1	Syntax and semantics	7
3.2	KEM-AC	8
3.2.1	Examples	8
3.3	TODO Live universe modifications	9
3.4	TODO Hybridization à la carte	10
4	TODO Optional key tracing	10
5	TODO Hierarchical Covercrypt and interoperability	10
6	TODO Implementation considerations	10
6.1	Key and encapsulation formats	10
6.1.1	Master secret key	11
6.1.2	Master public key	12
6.1.3	User secret key	13
6.1.4	Encapsulation	14
7	TODO Wish-list	15

1 Introduction

In large organizations, pieces of data created by one member are commonly shared to other members. However, some pieces of data are sensitive and must only be shared to a specific subset of members. The process of accessing a piece of data therefore becomes a three-step procedure requiring members to: (1) authenticate themselves to some trusted identity management system, (2) request the authorization to access this data from some trusted authorization management system and (3) request this data from the storage system which first verifies the validity of the identity claim delivered by the identity management system, the validity of the authorization claim delivered by the authorization management system, and the compatibility of those claims.

However, authorization rules may be complex which may lead to operator errors or bugs in the authorization and storage systems. Moreover actors may misbehave, which may question our trust in the authorization given by those systems. Hopefully, there are ways to raise the need for trust in those systems. One of them consists in cryptographically enforcing authorization by encrypting data under a key owned by the members of the targeted only. That way, the only trusted parties are the targeted members and the key distribution, which can be performed in a centralized Key Management System (KMS). But if authorization rules are complex, key distribution may also become complex and hence victim of the same kind of errors.

Attribute-Based Encryption (ABE) is a tool that has been extensively studied to solve exactly this problem. However, known ABE schemes are not very efficient and not post-quantum-secure which hinders their adoption. Covercrypt solves the same problem while relaxing some formal constraints in order to achieve efficiency in most practical use-cases while offering both CCA and post-quantum security, attribute-hiding ciphertexts and key tracing.

Covercrypt assumes the existence of a centralized KMS holding a master secret key with which all other keys are created: the master public key is used to encrypt data under its access policy while user secret keys are also associated to access policies and can decrypt all Covercrypt ciphertexts with a compatible access policy. Covercrypt follows the KEM-DEM approach in which data is symmetrically encrypted under a key which encapsulation is appended to the symmetric ciphertext. More precisely, Covercrypt is a actually Key-Encapsulation Mechanism with Access Control (KEM-AC) – as defined in 2.1 – that can be combined with a Data-Encryption Mechanism (DEM) to obtain a Public Key Encryption (PKE) scheme. In the rest of this document, we therefore prefer the term Covercrypt *encapsulation* to the term Covercrypt ciphertext. Moreover, when necessary, we use the terms *user policy* and *encapsulation policy* to denote an access policy associated to a user secret and a Covercrypt encapsulation respectively, and use the term *policy* the context makes it clear whether we refer to a user or encapsulation policy.

2 Subset-cover KEM-AC

2.1 Key-Encapsulation Mechanism with Access Control (KEM-AC)

A Key-Encapsulation Mechanism with Access Control (KEM-AC) allows encapsulating a secret with respect to an *encapsulation policy*, such that user secret key can open it if and only if its associated *user policy* is compatible with the encapsulation policy with respect to some binary relation over the policy space.

Definition 2.1 (Key-Encapsulation Mechanism with Access Control). A KEM-AC is defined over a master-secret-key space $\mathcal{K}_S$, master-public-key space $\mathcal{K}_P$ and a user-secret-key space $\mathcal{K}_U$, a session-key space $\mathcal{K}$ and an encapsulation space $\mathcal{E}$, and exposes the following operations:

setup: $\mathbb{P}(\mathcal{P}_U \times \mathcal{P}_E) \rightarrow \mathcal{K}_S \times \mathcal{K}_P$

This randomized algorithm takes as input a binary relation R over policies, and returns a master secret key msk_R and a master public key mpk_R .

keygen: $\mathcal{K}_S \times \mathcal{P}_U \rightarrow \mathcal{K}_U$

This randomized algorithm takes as input a master secret key $\text{msk}_R \in \mathcal{K}_S$ under a binary relation $R \subset \mathcal{P}_U \times \mathcal{P}_E$ and a policy $p_u \in \mathcal{P}_U$, and returns a user secret key $\text{usk}_{p_u, R}$.

encaps: $\mathcal{K}_P \times \mathcal{P}_E \rightarrow \mathcal{K} \times \mathcal{E}$

This randomized algorithm takes as input a master public key $\text{mpk}_R \in \mathcal{K}_P$ under a binary relation $R \subset \mathcal{P}_U \times \mathcal{P}_E$ and an encapsulation policy $p_e \in \mathcal{P}_E$, and returns both a session key $k \in \mathcal{K}$ and its encapsulation $\text{enc}_{p_e, R} \in \mathcal{E}$.

decaps: $\mathcal{K}_U \times \mathcal{E} \rightarrow \mathcal{K}$ or $\perp$

This algorithm takes as input a user secret key $\text{usk}_{p_u, R_u} \in \mathcal{K}_U$ associated to an access policy $p_u \in \mathcal{P}_U$ under a binary relation $R_u \subset \mathcal{P}_U \times \mathcal{P}_E$, and an encapsulation $\text{enc}_{p_e, R_e} \in \mathcal{E}$ associated to a policy $p_e \in \mathcal{P}_E$ under a binary relation $R_e \subset \mathcal{P}_U \times \mathcal{P}_E$. It returns the encapsulated session key $k \in \mathcal{K}$ if $(p_u, p_e) \in R_e$, and $\perp$ otherwise.

Covercrypt is a subset-cover algorithm: it implements a KEM-AC with respect to an access structure S defining a set of rights $\mathcal{U}$ called *universe*, which policy space is $\mathbb{P}(\mathcal{U})$ and binary relation is the test of intersection: $R_{\mathcal{U}} = \{(p_u, p_e) \in \mathbb{P}(\mathcal{U})^2 : p_u \cap p_e \neq \emptyset\}$. More precisely, the Covercrypt setup is as follows (the other primitives remain the same):

setup: $S \rightarrow \mathcal{K}_S \times \mathcal{K}_P$

This randomized algorithm takes as input an access structure S defining a universe $\mathcal{U}$, and returns a master secret key $\text{msk}_{R_{\mathcal{U}}}$ and a master public key $\text{mpk}_{R_{\mathcal{U}}}$.

2.2 Key and encapsulation life-cycle management

Just like any other keys, Covercrypt keys can be compromised and lost. There is therefore the need to renew them. However, in order to preserve the ability to open older encapsulations with renewed keys, it is desirable for the key renewal to be different from setting up a new instance. The renewal procedure – called *key rotation* – is described in the present section.

Recall that each user secret key usk_p is associated to a policy p . Upon use-secret-key compromise, a key rotation with respect to its entire policy must be performed. Due to the particular structure of Covercrypt keys, this process implies: (1) the rotation of all associated secrets in the master secret key, (2) the propagation of this rotation to the master public key to prevent future encapsulations from being generated with respect to compromised secrets, (3) the propagation of this rotation to all user secret keys which policy intersects the ones of the compromised key to allow legitimate users opening the new encapsulations generated with respect to the new secrets, and (4) optionally the refresh of all encapsulations which associated policy intersects the one of the compromised key to prevent an eventual attacker from opening them if it still hasn't accessed them. The entire key-rotation process is depicted in Figure 1, and leverages the following primitives:

usk-policy: $\mathcal{K}_U \rightarrow \mathbb{P}(\mathcal{U})$

This function takes as input a Covercrypt user secret key $usk_p \in \mathcal{K}_U$ and returns its associated policy $p \in \mathcal{U}$.

rotate: $\mathcal{K}_S \times \mathbb{P}(\mathcal{U}) \rightarrow \mathcal{K}_S \times \mathcal{K}_P$

This randomized algorithm takes as input a master secret key $msk_{\mathcal{U}} \in \mathcal{K}_S$ and an access policy $p^* \in \mathcal{U}$, refreshes every secrets in this given master secret key that are associated to a right in this policy (*i.e.* each secret associated to a right in $s \cap p^*$), and returns the refreshed master secret key along with its associated master public key.

usk-refresh: $\mathcal{K}_S \times \mathcal{K}_U \rightarrow \mathcal{K}_U$ or $\perp$

This algorithm takes as input a master secret key $msk \in \mathcal{K}_S$ and a user secret key $usk_p \in \mathcal{K}_U$ associated to a policy $p \in \mathcal{U}$. If the integrity of the given user secret key can be verified by the master secret key, it returns a new user secret key usk'_p associated to the same access policy containing all secrets that are contained in the given user secret key, plus new secrets for each rights in p for which newer secrets exists in the master secret key. If the integrity of the given user secret key cannot be verified, $\perp$ is returned.

enc-refresh: $\mathcal{K}_S \times \mathcal{K}_P \times \mathcal{E} \rightarrow \mathcal{E}$ or $\perp$

This randomized algorithm takes as input a master secret key $msk \in \mathcal{K}_S$, a master public key $mpk \in \mathcal{K}_S$ and an encapsulation $enc_p \in \mathcal{E}$ associated to a policy $p \in \mathcal{U}$. If the integrity of the master public key can be verified, that its revision number matches that of the master secret key, and that the encapsulation can be open, it returns a new encapsulation enc'_p associated to the same access policy. Otherwise $\perp$ is returned.

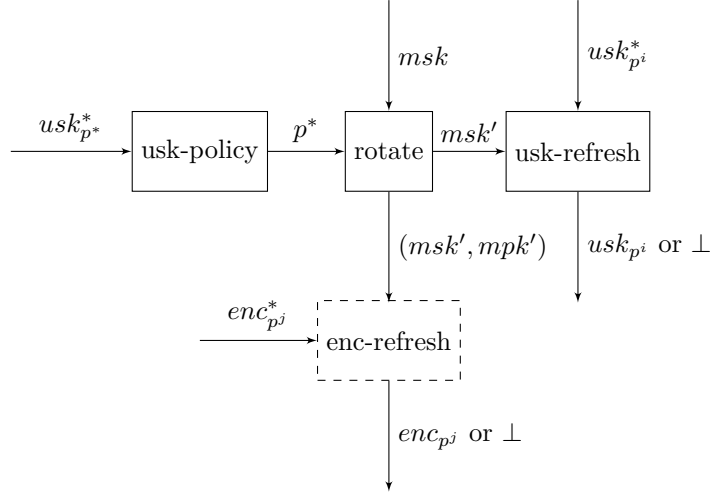


Figure 1: User-secret-key rotation flow, where untrusted objects are marked with an asterix.

Remark. Checking the integrity of the user secret key (resp. encapsulation key) upon refresh is required to prevent right escalation (resp. impersonification) via modification of this key, while encapsulation policies cannot be modified via modification of the encapsulation due to the CCA security guaranteed by Covercrypt encapsulations.

Additionally, to prevent impersonification attacks upon encapsulation, users can always verify the integrity of the master public key using user secret keys, and since this is necessary to prevent right-escalation attacks upon user-secret-key refresh, we also provide a way to verify the integrity of user secret keys using the master secret key:

mpk-verify: $\mathcal{K}_U \times \mathcal{K}_P \rightarrow \mathcal{B}$

Returns whether the integrity of the given master public key $mpk \in \mathcal{K}_P$ can be verified using the given user secret key $usk_p \in \mathcal{K}_U$.

usk-verify: $\mathcal{K}_S \times \mathcal{K}_P \rightarrow \mathcal{B}$

Returns whether the integrity of the given user secret key $usk_p \in \mathcal{K}_U$ can be verified using the given master secret key $msk \in \mathcal{K}_S$.

Finally, a Covercrypt instantiation is associated to a sequence of *revisions*: the setup initializes an instance with *revision number* 0, and each execution of the key rotation procedure increments this number by one and returns the new value of the master secret key with the new revision number. Covercrypt objects can only create objects with the same revision number which can be extracted via one of the following functions:

msk-revision: $\mathcal{K}_S \rightarrow \mathbb{N}$

Retrieves the revision number from the given master secret key $msk \in \mathcal{K}_S$.

mpk-revision: $\mathcal{K}_P \rightarrow \mathbb{N}$

Retrieves the revision number from the given master public key $mpk \in \mathcal{K}_P$.

usk-revision: $\mathcal{K}_U \rightarrow \mathbb{N}$

Retrieves the revision number from the given master public key $\text{usk} \in \mathcal{K}_U$.

enc-revision: $\mathcal{E} \rightarrow \mathbb{N}$

Retrieves the revision number from the encapsulation $\text{enc} \in \mathcal{E}$.

Remark. Downgrade attacks upon encapsulation can be avoided by verifying the integrity of the master public key used, and that its revision number is no less than that of the user secret key.

2.3 TODO Hybridization à la carte

To accommodate different regulations and right sensitivities, it is possible to describe for each right the kind of encapsulation to generate, among a set of predefined encapsulation kinds.

2.4 Live access-structure modifications

Real-life access structure may evolve with time. Covercrypt reflects this by exposing primitives allowing to modify the policy universe without having to perform a new setup. In particular, additive modifications of the universe are *backward-compatible*: old user secret keys and encapsulations need not be modified in order to open new compatible encapsulations, or to be opened by new compatible user secret keys. In other words, additive universe modifications can be seen as the use of a right that was present in the universe but unused until then.

add-rights: $\mathcal{K}_S \times \mathbb{P}(\mathcal{R}) \rightarrow \mathcal{K}_S \times \mathcal{K}_P$

This randomized algorithm takes as input a master secret key $\text{msk} \in \mathcal{K}_S$ under a universe $\mathcal{U}$ and a set of new rights $\{r_i\}_i \subset \mathcal{R}$, and returns new master secret and public keys under a universe $\mathcal{U}' \leftarrow \mathcal{U} \cup \{r_i\}_i$.

del-rights: $\mathcal{K}_S \times \mathbb{P}(\mathcal{R}) \rightarrow \mathcal{K}_S \times \mathcal{K}_P$

This randomized algorithm takes as input a master secret key $\text{msk} \in \mathcal{K}_S$ under a universe $\mathcal{U}$ and a set of new rights $\{r_i\}_i \subset \mathcal{R}$, and returns new master secret and public keys with incremented revision numbers under a universe $\mathcal{U}' \leftarrow \mathcal{U} \setminus \{r_i\}_i$.

Remark. Additive universe modifications do not generate new revisions while destructive universe modifications generate new revisions since downgrade attacks can leverage master public keys prior to the removal of some rights from the universe to trick users create encapsulations for compromised secrets.

3 Boolean KEM-AC

In order to make expressing policies more ergonomic, we introduce KEM-AC which policy space is a set of logical propositions that is closed under the two logical operators *and* ($\wedge$) and *or* ($\vee$). Then, we show how to implement this boolean KEM-AC on top of the set-based KEM-AC defined in Section 2 via compiling boolean policies to set-based policies.

```

<policy>      ::= <attribute> | <policy> <operator> <policy>
<operator>    ::= "&&" | "||"
<attribute>   ::= "*" | <dimension>::<name>
<dimension>   ::= <name>
<name>        ::= <word> \ { "&&" | "||" | <word>::<word> }
<word>        ::= char+

```

Figure 2: Syntax of the policy language.

3.1 Syntax and semantics

The policy language has a very simple syntax which specification is given in Figure 2, and which semantics is defined thereafter.

Definition 3.1 (Universe). In the boolean KEM-AC setting, the universe Ω is a set of atomic logical propositions called *attributes*, and the universe of expressible attributes $\mathcal{U}$ is a subset of the universe Ω .

Definition 3.2 (Policies). The space of policies $\mathcal{P}_\Omega$ associated to a universe of attributes Ω is the closure of Ω with respect to both logical operators *and* ($\wedge$) and *or* ($\vee$), noted $\mathcal{P}_\Omega = \langle \Omega, \wedge, \vee \rangle$. This set has two special policies, noted **True** (resp. **False**), that are always true (resp. false).

Definition 3.3 (Implication in $\mathcal{U}$). Given two logical propositions $(p, p') \in \mathcal{P}_\Omega^2$ and a universe of expressible attributes $\mathcal{U} \subset \Omega$, p is said to imply p' in $\mathcal{U}$, noted $p \implies_{\mathcal{U}} p'$, if and only if p implies p' when restricted to $\mathcal{U}$:

$$\forall (p_u, p_e) \in \mathcal{U}^2, (p \implies_{\mathcal{U}} p') \iff (p|_{\mathcal{U}} \implies p'|_{\mathcal{U}})$$

TODO: I feel like I want to come back to the view that the implication in $\mathcal{U}$ is the implication *knowing* that all attributes outside of $\mathcal{U}$ are set to **false**.

In order to structure the set of expressible policies, logically-related attributes can be grouped by dimension, which come in two kinds:

Definition 3.4 (Dimension). A dimension D of a universe of attributes Ω is a non-empty set of attributes from Ω such that at least one of its attributes is true. In other words, the set of dimensions $\mathcal{D}$ is:

$$\mathcal{D} = \left\{ \{a_i\} \in \mathbb{P}(\Omega)^* : \bigvee_i a_i = \text{True} \right\}$$

TODO: we need to add the fact that attributes from different dimensions are *not* exclusive.

Definition 3.5 (Hierarchy). A hierarchical dimension of a universe $\mathcal{U}$, noted $D \in \mathcal{H}_{\mathcal{U}}$, is a dimension of $\mathcal{U}$ which attributes are strictly increasing in $\mathcal{U}$:

$$\mathcal{H}_{\mathcal{U}} = \left\{ D \in \mathcal{D}_{\mathcal{U}} \mid \exists a \in D : \begin{array}{l} \forall a' \in D, a \implies_{\mathcal{U}} a' \\ \forall a' \in D, a \neq a' \implies \neg(a' \implies_{\mathcal{U}} a) \\ (D = \{a\}) \vee (D \setminus \{a\} \in \mathcal{H}_{\mathcal{U}}) \end{array} \right\} \begin{array}{l} \text{ordered} \\ \text{strictly} \\ \text{totally} \end{array}$$

In other words, the logical implication in $\mathcal{U}$ defines a strict and total order relation over hierarchical dimensions, and in particular, for each hierarchical dimension $D \in \mathcal{H}_{\mathcal{U}}$, there exists a smallest attribute $a_{\perp}^D \in D$ that is always true in $\mathcal{U}$: $a_{\perp}^D = \text{True}$.

Definition 3.6 (Anarchy). An anarchical dimension of a universe $\mathcal{U}$, noted $D \in \mathcal{A}_{\mathcal{U}}$, is a dimensions of $\mathcal{U}$ containing mutually exclusive propositions:

$$\mathcal{A}_{\mathcal{U}} = \{D \in \mathcal{D}_{\mathcal{U}} \mid \forall (a, a') \in D^2, a = a' \vee (a \implies_{\mathcal{U}} \neg a')\}$$

Not all combinations of dimensions can be used within a given Covercrypt instance. The access structure associated to the master secret and public keys defines which dimensions are expressible, and must respect the constraint that expressible dimensions are mutually disjoint.

Definition 3.7 (Access Structure). In the boolean KEM-AC setting, an access structure is a set of mutually disjoint dimensions:

$$\mathcal{S} = \{S \in \mathbb{P}(\mathcal{D}) \mid \forall (D_i, D_j) \in S^2, D_i \cap D_j = \emptyset\}$$

Definition 3.8 (Valid Policy). A policy p is said to be valid with respect to a set of expressible attributes $\mathcal{U}$, noted $p \in \mathcal{V}_{\mathcal{U}}$ if and only if p restricted to $\mathcal{U}$ is not the constant policy **False**:

$$p \in \mathcal{V}_{\mathcal{U}} \iff p =_{\mathcal{U}} \text{False}$$

3.2 KEM-AC

In the boolean KEM-AC setting, a user secret key $\text{usk}_{p_u, \mathcal{U}}$ associated to a policy $p_u \in \mathcal{P}_{\mathcal{U}}$ can open an encapsulation $\text{enc}_{p_e, \mathcal{U}'}$ associated to a policy $p_e \in \mathcal{P}_{\mathcal{U}'}$ if and only if the conjunction of p_u and p_e is valid in both $\mathcal{U}$ and $\mathcal{U}'$:

$$(p_u, p_e) \in R \iff (p_u \wedge p_e) \in (\mathcal{V}_{\mathcal{U}} \cap \mathcal{V}_{\mathcal{U}'})$$

Remark. It is important to distinguish the universe in which an encapsulation was created from the one in which a user secret key was created since the universe of expressible attributes can be both extended and expanded in such a way that a access policy that is valid in a first universe $\mathcal{U}$ may not be valid in the new universe $\mathcal{U}'$.

3.2.1 Examples

1. Without expressible universe modification

Let $\mathcal{U} = D_1 \cup D_2 \subset \Omega$, where $D_1 = \{a_1^1, a_2^1, a_3^1\} \subset \mathcal{A}_{\Omega}$ is an anarchical dimension, and $D_2 = \{a_1^2, a_2^2\} \subset \mathcal{H}_{\Omega}$ is a hierarchical dimension with $a_2^2 \implies_{\mathcal{U}} a_1^2$.

- Let $p_u = a_1^1 \wedge a_2^2$ and $p_e = a_1^1 \wedge a_1^2$. In this case, a user secret key $\text{usk}_{p_u, \mathcal{U}}$ can open an encapsulation $\text{enc}_{p_e, \mathcal{U}}$ since $p_u \wedge p_e = a_1^1 \wedge a_2^2 \neq \text{False}$.

- Let $p_u = a_1^1 = a_1^1 \wedge a_1^2$ using the definition of a hierarchical dimension. Let $p_e = a_2^2 = a_2^2 \wedge (\bigvee_{i=1}^3 a_i^1) = \bigvee_{i=1}^3 (a_2^2 \wedge a_i^1)$ using the definition of an anarchical dimension. In this case, a user secret key $\text{usk}_{p_u, \mathcal{U}}$ *cannot* open an encapsulation $\text{enc}_{p_e, \mathcal{U}}$ since $\neg(a_1^2 \implies_{\mathcal{U}} a_2^2)$ by definition of a hierarchical dimension.

2. With expressible universe modification

(a) Additive transformation

Let $\mathcal{U}' = \mathcal{U} \cup D_3$, where $D_3 = \{a_1^3, a_2^3\} \subset \mathcal{A}_\Omega$ is an anarchical dimension.

- Let $p_u = a_1^1 \wedge a_2^2$ and $p_e = a_1^1 \wedge a_2^2 \wedge a_2^3$. Then a user secret key $\text{usk}_{p_u, \mathcal{U}}$ *cannot* decrypt $\text{enc}_{p_e, \mathcal{U}'}$ since $p_e =_{\mathcal{U}} \text{False}$ and $\neg(p_u \implies_{\mathcal{U}} \text{False})$.
 $p_u =_{\mathcal{U}'} a_1^1 \wedge a_2^2 \wedge (a_1^3 \vee a_2^3)$
- Let $p_u = a_1^1 \wedge a_2^2$ and $p_e = a_1^1 \wedge a_2^2 \wedge a_2^3$. Then a user secret key $\text{usk}_{p_u, \mathcal{U}}$ *cannot* decrypt $\text{enc}_{p_e, \mathcal{U}'}$ since

(b) Destructive transformation

3.3 TODO Live universe modifications

Evolving the access structure underlying a given revision of a Covercrypt instance can be done through one of those functions:

add-hierarchy: $\mathcal{K}_S \times \mathbb{P}(\mathcal{R}) \rightarrow \mathcal{K}_S \times \mathcal{K}_P$ or $\perp$

del-hierarchy: $\mathcal{K}_S \times \mathbb{P}(\mathcal{R}) \rightarrow \mathcal{K}_S \times \mathcal{K}_P$ or $\perp$

This randomized algorithm takes as input a master secret key $msk \in \mathcal{K}_S$ with an access structure S and a set of attributes $D \subset \mathcal{R}$. If this new hierarchy is valid with respect to S , it returns a new master secret key and a new master public key which structure contains (resp. does not contain) H . Otherwise $\perp$ is returned.

add-anarchy: $\mathcal{K}_S \times \mathcal{A} \rightarrow \mathcal{K}_S \times \mathcal{K}_P$ or $\perp$

del-anarchy: $\mathcal{K}_S \times \mathcal{A} \rightarrow \mathcal{K}_S \times \mathcal{K}_P$ or $\perp$

This randomized algorithm takes as input a master secret key $msk \in \mathcal{K}_S$ with an access structure S and an anarchical dimension $A \in \mathcal{A}$. If this new anarchy is valid with respect to S , it returns a new master secret key and a new master public key which structure contains (resp. does not contain) A . Otherwise $\perp$ is returned.

add-attribute: $\mathcal{K}_S \times \mathcal{A} \times \mathcal{U} \rightarrow \mathcal{K}_S \times \mathcal{K}_P$ or $\perp$

del-attribute: $\mathcal{K}_S \times \mathcal{A} \times \mathcal{U} \rightarrow \mathcal{K}_S \times \mathcal{K}_P$ or $\perp$

This randomized algorithm takes as input a master secret key $msk \in \mathcal{K}_S$ with an access structure S , an anarchical dimension $D \in \mathcal{A}$ and an attribute $p \in \mathcal{U}$. If this dimension is a member of S , it returns a new master secret key and a new master public key with an access structure in which this dimension contains (resp. does not contain) this attribute.

add-hierarchical-attribute: $\mathcal{K}_S \times \mathcal{H} \times \mathcal{U} \rightarrow \mathcal{K}_S \times \mathcal{K}_P$ or $\perp$

del-hierarchical-attribute: $\mathcal{K}_S \times \mathcal{H} \times \mathcal{U} \rightarrow \mathcal{K}_S \times \mathcal{K}_P$ or $\perp$

This randomized algorithm takes as input a master secret key $msk \in \mathcal{K}_S$ with an access structure S , a hierarchical dimension $D \in \mathcal{H}$ and an attribute $p \in \mathcal{U}$. If this dimension is a member of S , it returns a new master secret key and a new master public key with an access structure in which this dimension contains (resp. does not contain) this attribute.

3.4 TODO Hybridization à la carte

4 TODO Optional key tracing

5 TODO Hierarchical Covercrypt and interoperability

6 TODO Implementation considerations

6.1 Key and encapsulation formats

Covercrypt is parameterized by:

- a signature scheme that could be described by the following signature:

```
module type Signature = sig
  type sk
  type vk
  type signature

  val sign : sk -> msg:string -> signature
  val verf : vk -> msg:string -> signature -> bool
end
```

- a mac scheme that could be described by the following signature:

```
module type MAC = sig
  type key
  type tag

  val mac : key -> msg:string -> tag
end
```

- a key-homomorphic Non-Interactive Key-Exchange (NIKE) scheme that could be described as:

```
module type Group = sig
  type t

  val id : t
```

```

    val op : t -> t -> t
    val inv : t -> t
end

module type AbelianGroup = sig
    include Group

    val zero : t
    val ( + ) : t -> t -> t
    val ( - ) : t -> t -> t
end

module type Ring = sig
    include AbelianGroup

    val one : t
    val ( * ) : t -> t -> t
    val ( / ) : t -> t -> t option
end

module type KH_NIKE = sig
    module Sk : Ring
    module Pk : Group

    type key
    type rng

    val op : Pk.t -> Sk.t -> Pk.t
    val keygen : rng -> Sk.t * Pk.t
    val session : Pk.t -> Sk.t -> key
end

```

Such that: `session (op pk sk) sk' = session pk Ring.(sk * sk')`.

- a KEM that could be described by the following signature:

```

module type KEM = sig
    type dk
    type ek
    type ss
    type enc
    type rng

    val keygen : rng -> dk * ek
    val encaps : rng -> ek -> ss * enc
    val decaps : dk -> enc -> ss
end

```

6.1.1 Master secret key

The master secret key is composed of:

1. a revision number v ;
2. a signing key sk used to sign the master public key;
3. a mac key k used to verify the integrity of user secret keys;
4. a tracing secret key tsk ;
5. a mapping rsk from rights to histories of secrets.

```

module Covercrypt_msk
  (S : Signature)
  (M : MAC)
  (Nike : KH_NIKE)
  (Kem : KEM)
  (Right : sig type t end)
= struct
  type msk = {
    v : int;
    k : M.key;
    sk : S.sk;
    tsk : tsk;
    rsk : rsk;
  }
  and tsk = Nike.Sk.t list
  and rsk = (Right.t, secret list) Hashtbl.t
  and secret =
    | N of Nike.Sk.t
    | K of Kem.dk
    | H of Nike.Sk.t * Kem.dk
end

```

With the additional constraints that there exist a tracing threshold $t \in \mathbb{N}^*$ such that $|tsk| = t + 2$.

6.1.2 Master public key

The master public key is composed of:

1. a revision number v ;
2. a tracing public key tpk ;
3. a mapping rp_k from the right to a public element;
4. a signature σ of its other components under the signing key of the master secret key;

```

module Covercrypt_mpk
  (S : Signature)
  (Nike : KH_NIKE)
  (Kem : KEM)
  (Right : sig type t end)

```

```

= struct
  type mpk = {
    v : int;
    tpk : tpk;
    rpkm : rpkm;
    sgn : S.signature;
  }
  and tpk = Nike.Pk.t list
  and rpkm = (Right.t, elt list) Hashtbl.t
  and elt =
    | N of Nike.Pk.t
    | K of Kem.ek
    | H of Nike.Pk.t * Kem.ek
end

```

With the additional constraints that there exist a tracing threshold $t \in \mathbb{N}^*$ such that $|tpk| = t + 2$.

6.1.3 User secret key

A user secret key is composed of:

1. a revision number v ;
2. a verification key vk to verify the master public key;
3. a tracing user key tuk ;
4. a mapping ruk of rights onto histories of secrets;
5. a MAC tag of its other components.

```

module Covercrypt_usk
  (S : Signature)
  (M : MAC)
  (Nike : KH_NIKE)
  (Kem : KEM)
  (Right : sig type t end)
= struct
  type usk = {
    v : int;
    vk : S.vk;
    tuk : tuk;
    ruk : ruk;
    tag : M.tag
  }
  and tuk = Nike.Sk.t list
  and ruk = (Right.t, secret list) Hashtbl.t
  and secret =
    | N of Nike.Sk.t
    | K of Kem.dk
    | H of Nike.Sk.t * Kem.dk
end

```

With the additional constraints that there exist a tracing threshold $t \in \mathbb{N}^*$ such that $|tuk| = t + 1$.

6.1.4 Encapsulation

An encapsulation is composed of:

1. a revision number v ;
2. a verification tag;
3. a tracing encapsulation $tenc$;
4. an attribute-hiding access-control encapsulation $renc$;

```

module Sized_string = struct
  module type S = sig
    type t
    val size : int
    val to_string : t -> string
    val of_string : string -> t option
  end

  module Make (P : sig val size : int end) : S = struct
    type t = string
    let size = P.size
    let to_string s = s
    let of_string s =
      if String.length s = size
      then Some s else None
    end
  end

  module Covercrypt_enc
    (Nike : KH_NIKE)
    (Kem : KEM)
  = struct
    module VTag = Sized_string.Make (struct let size = 16 end)
    module NEnc = Sized_string.Make (struct let size = 32 end)
    type enc = {
      v : int;
      tag : VTag.t;
      tenc : Nike.Pk.t list;
      renc : renc;
    }
    and renc =
      | N of NEnc.t list
      | K of Kem.enc list
      | H of NEnc.t * Kem.enc
  end

```

With the additional constraints that there exist a tracing threshold $t \in \mathbb{N}^*$ such that $|tenc| = t + 1$.

7 TODO Wish-list